Parity System Map

Three TypeScript projects, one SQLite file, and a study that scanned 92,290 prediction markets on two platforms looking for a cross-platform arbitrage and found zero. This is how the pieces connect, what the numbers are, and how the public site runs with no backend at all.

Polymarket · Kalshi · Confirmed arbitrage, reserved and unused · Engine, API, app, files

26,930
Polymarket markets, a lower bound

65,360
Kalshi markets

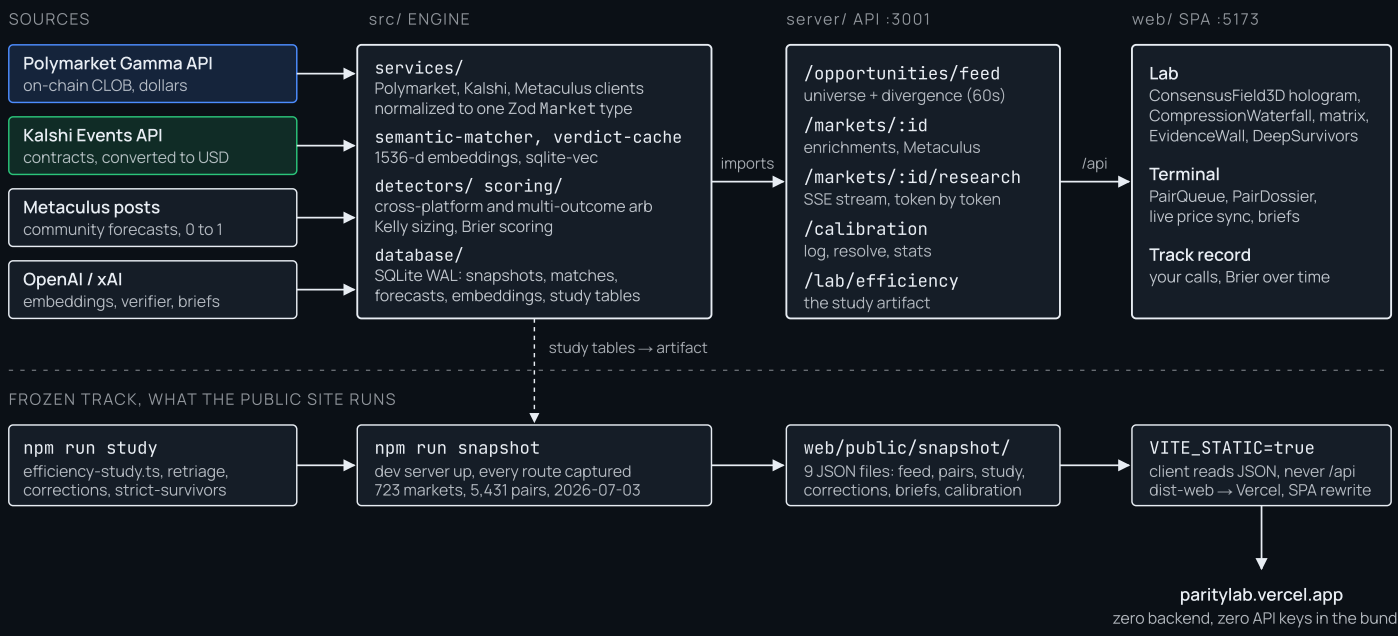
11,138
cached verifier verdicts, one model, one prompt

0
clear executable arbitrages after eleven gates

1 · THE SYSTEM

Two tracks: live and frozen

The same React app runs two ways. Locally it talks to a Hono API over `/api`. In production it reads nine JSON files frozen by `npm run snapshot`, so the public site is a static bundle on Vercel with nothing to keep alive.



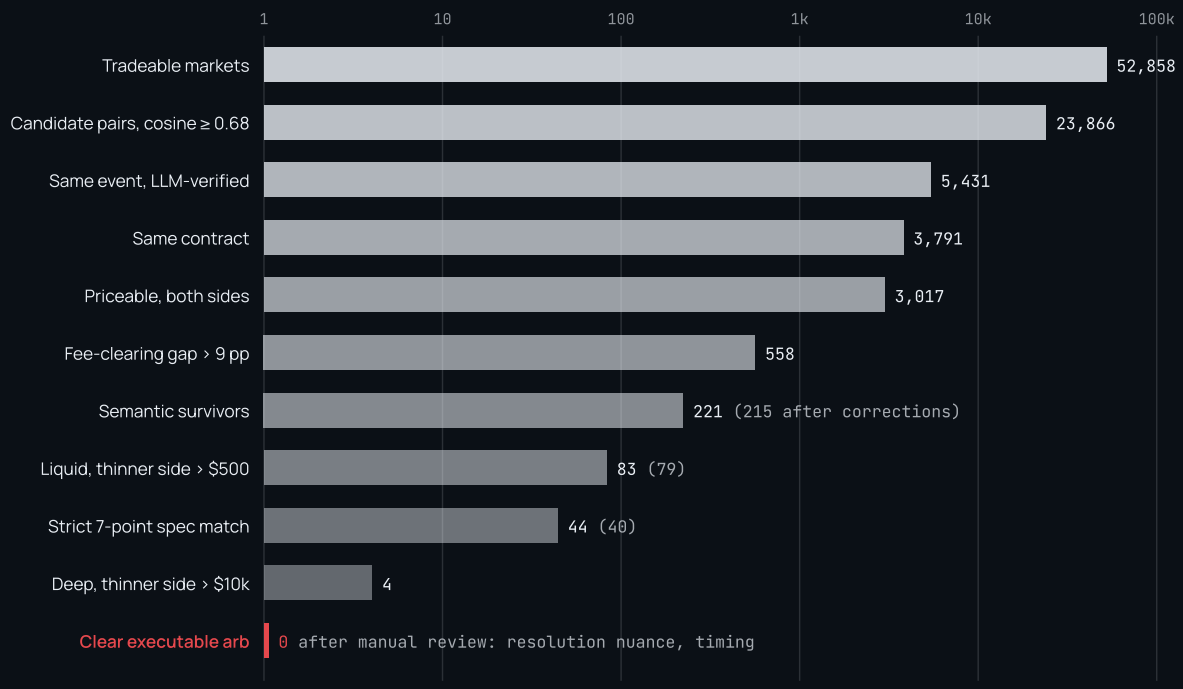
Three tsconfigs typecheck independently; the engine has no React or HTTP dependency and is shared by the API, the CLI dashboard and the scheduler.

Top row is the live system. Bottom row is the frozen track: the study runs, the snapshot captures every response, Vite bundles the JSON, and the public site never makes a network call to anything but its own files.

2 · THE STUDY

Eleven gates from 52,858 markets to zero

The waterfall the Lab draws, with the frozen counts. Widths are on a log scale because the last six stages would be invisible next to the first; every bar carries its count.



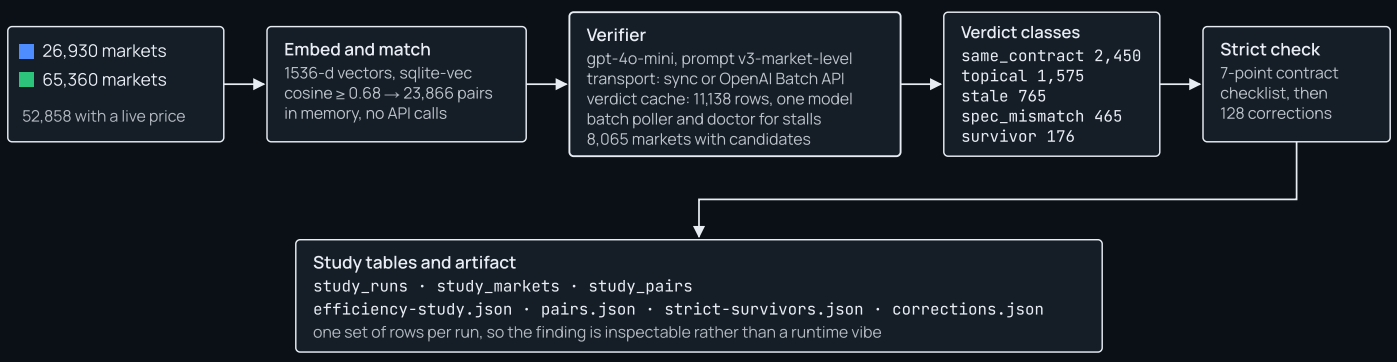
Counts in parentheses apply the correction overlay: 128 pairs the strict re-check or a deterministic scope rule reclassified, e.g. a county race matched to the statewide race.

Each stage is a gate with a stated reason, never an arbitrage claim. Executability was never demonstrated because no depth, slippage or timing was tested; the zero is the absence of a demonstrable edge.

3 · THE VERIFIER

How pairs are matched, judged and corrected

Matching is cheap and local; judgment is a language model with every verdict cached; corrections are the model auditing its own false positives.

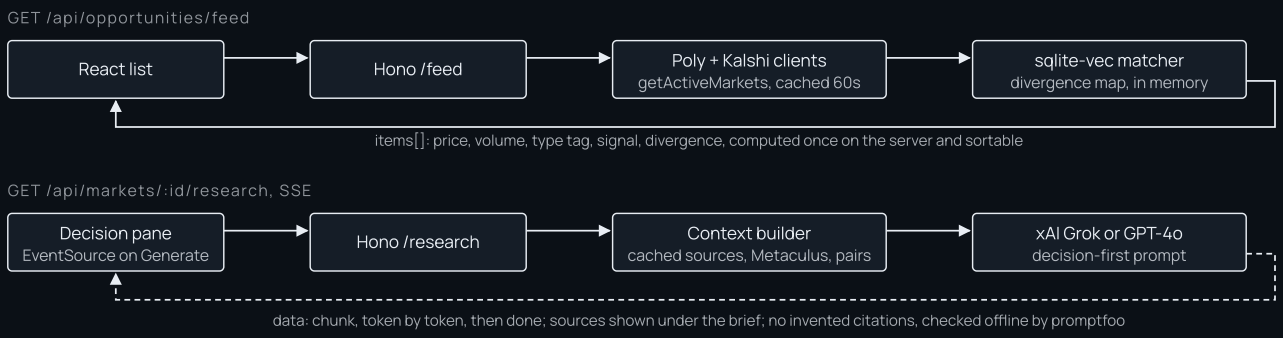


The same detector pattern lives in two places on purpose, web/src/lib/pairStatus.ts and scripts/build-corrections.ts, and they must stay in sync.

Left to right: two universes become one embedded space, candidates get a cached model verdict, verdicts get classified, then a stricter check and a correction overlay produce the files the Lab and Terminal render.

4 · TWO REQUEST LIFECYCLES

The list and the brief



The list is computed once per request on the server so its columns sort. The brief is on demand and streams, with its context gathered from files collected ahead of time, never scraped live.

5 · THE CODE

Where things live

PATH	WHAT IT IS	FILES
`src/services/`	platform clients, semantic and cross-platform matchers, embedding cache, verdict cache, Batch verifier, settlement comparator	12
`src/detectors/, scoring/, database/`	arbitrage and divergence detectors; Kelly and Brier scoring; schema, queries, study store	12
`server/src/routes/`	opportunities, markets, research (SSE), calibration; plus cache, pairs-data, prompts	4
`web/src/pages/`	LabPage, TerminalPage	2
`web/src/components/lab/`	ConsensusField3D hologram, CompressionWaterfall, ComparisonMatrix, EvidenceWall, DeepSurvivors	8
`web/src/lib/`	funnel.ts (the eleven stages, one place), pairStatus.ts (the detector), verifier index, count-up	8
`scripts/`	efficiency-study, retriage, build-corrections, strict-survivors, snapshot, verify-batch, verify-batch-doctor, collect-market-context	12
`web/public/snapshot/`	the nine frozen JSON responses the public site runs on	9

`npm run check` is the gate before any push: three typechecks, the Vite build, and 435 tests. `npm run study` regenerates the artifact; `npm run snapshot` freezes it; Vercel builds with `VITE_STATIC=true` from the committed snapshot. Colors are locked everywhere: blue is Polymarket, green is Kalshi, red is reserved for a confirmed arbitrage and has never been used.